图灵知识图谱 — Turing Knowledge Graph

> 知识工程课程作业：以艾伦·图灵（Alan Turing）为核心，从网络采集数据并构建领域知识图谱。

一、项目简介

本项目旨在围绕"计算机科学之父"**艾伦·图灵**，端到端地完成知识图谱的构建流程，覆盖以下核心环节：

模块	说明
数据采集	从维基百科（Wikipedia）自动爬取图灵及其关联页面的结构化 / 非结构化信息
实体识别与消歧	基于 `spaCy` 预训练 NER 模型识别多类型实体，并通过 Wikidata 实体链接完成消歧
关系抽取	从文本中抽取实体间的语义关系，形成三元组
知识图谱构建	将三元组存储为有向图结构，支持增量更新与持久化
可视化	生成多视角的图谱可视化图片及交互式页面
图谱查询	基于最终图谱提供关键词检索、节点详情、局部子图、Web API 与前端动态展示

二、项目结构

```
KG-Turing/
├── README.md                        # 项目说明文档（本文件）
├── requirements.txt                 # Python 依赖清单
├── config/
│   ├── relation_mapping.yaml        # Infobox 键到标准关系标签的映射
│   └── settings.yaml                # 爬虫与路径配置
├── scripts/
│   ├── check_deps.py                # 关键依赖、torch/CUDA、spaCy 模型与可选依赖检查
│   ├── check_torch.py               # PyTorch / CUDA 环境检查
│   └── run_pipeline.py              # 关系抽取 → 图构建 → 可视化一键脚本（演示 5 篇文档）
│   └── run_webapp.py                # 启动查询与前端界面（Flask / 标准库双模式）
├── src/
│   ├── __init__.py
│   ├── data_extraction/
│   │   ├── __init__.py
│   │   ├── wiki_crawler.py          # BFS 爬取页面并写入 data/raw
│   │   ├── wiki_parser.py           # 解析单个 Wikipedia 页面
│   │   ├── data_cleaner.py          # 清洗文本并切分句子
│   │   └── run_extraction.py        # 数据采集总入口
│   ├── ner/
│   │   ├── __init__.py
│   │   ├── spacy_ner.py             # spaCy NER 封装
│   │   ├── entity_linker.py         # Wikidata 实体消歧
│   │   ├── ner_pipeline.py          # 文本级 NER + 链接接口
│   │   └── batch_process.py         # 目录级批处理，输出 entities_all.jsonl
│   ├── relation_extraction/
│   │   ├── __init__.py              # 关系抽取模块初始化
│   │   ├── extract_infobox_triples.py # 从 Infobox 抽取高置信三元组
│   │   ├── generate_candidates.py    # 基于实体识别结果生成句内候选实体对
│   │   ├── build_silver_labels.py    # 用 Infobox 远程监督构建银标关系样本
│   │   ├── rebel_extract.py          # 调用 REBEL 模型抽取文本三元组并对齐候选实体
│   │   ├── merge_triples.py          # 归一化谓词并合并 Infobox/Silver/REBEL 三元组
│   │   └── apply_aliases.py          # 标准化实体别名，减少重复节点
│   ├── kg_construction/
│   │   ├── __init__.py
│   │   └── build_graph.py           # 三元组 → NetworkX 图谱
│   ├── query/
│   │   ├── __init__.py
│   │   └── graph_query.py           # 图谱关键词检索、节点详情与子图提取
│   └── visualization/
│       ├── __init__.py
```

```
│       └── visualize.py            # 静态图、交互图、Ego 子图
│   └── webapp/
│       ├── __init__.py
│       ├── app.py                  # Web API、脚本调度与下载接口
│       ├── templates/
│       │   └── index.html          # 控制台页面模板
│       └── static/
│           ├── app.js              # 前端交互与动态图谱渲染
│           └── style.css           # 前端样式
├── data/
│   ├── raw/                        # 爬取后的原始页面 JSON
│   ├── processed/                  # 清洗并分句后的页面 JSON
│   └── relation/                   # 关系抽取中间文件（候选对、银标等）
├── output/
│   ├── entities_all.jsonl          # 文档级实体识别与消歧结果
│   ├── graphs/                     # 三元组与图谱文件
│   ├── logs/                       # REBEL 与合并日志 / 质检结果
│   └── visualizations/             # PNG / HTML 可视化结果
├── lib/                            # 前端依赖与静态资源（vis-network、tom-select 等）
└── tests/
  ├── test_ner.py                 # NER 单元测试
  ├── test_graph_query.py         # 图谱查询服务测试
  └── test_webapp_api.py          # Web API 测试
```

2.1 当前代码实现的整体工作流程

项目已经打通从 Wikipedia 页面抓取到知识图谱查询与前端展示的完整链路。当前代码对应的端到端流程如下：

```
可选环境检查
  ├ scripts/check_deps.py
  └ scripts/check_torch.py

数据采集
  src/data_extraction/run_extraction.py
    ├ wiki_crawler.py      负责 BFS 爬取页面标题
    ├ wiki_parser.py       负责解析单页 HTML / API 数据
    └ data_cleaner.py      负责清洗文本与分句
  输出：data/raw/*.json -> data/processed/*.json

实体识别与消歧
  src/ner/batch_process.py
    ├ spacy_ner.py         负责实体识别
    ├ entity_linker.py     负责 Wikidata 链接
    └ ner_pipeline.py      提供轻量函数封装
  输出：output/entities_all.jsonl

关系抽取
  1. extract_infobox_triples.py  -> output/graphs/infobox_triples.jsonl
  2. generate_candidates.py      -> data/relation/candidates.jsonl
  3. build_silver_labels.py      -> data/relation/silver.jsonl
  4. rebel_extract.py            -> output/graphs/rebel_triples.jsonl
  5. merge_triples.py            -> output/graphs/relation_triples.jsonl
  6. apply_aliases.py            -> output/graphs/relation_triples_aliased.jsonl

知识图谱构建
  src/kg_construction/build_graph.py
  输出：output/graphs/knowledge_graph.graphml / .gexf / .json

图谱可视化
  src/visualization/visualize.py
  输出：output/visualizations/full_graph.png / full_graph.html / ego_*.png / ego_*.html

图谱查询与 Web 展示
  src/query/graph_query.py
    ├ 关键词检索、节点详情、局部子图
    └ 直接消费 output/graphs/knowledge_graph.json
  scripts/run_webapp.py / src/webapp/app.py
```

```
    ├─ Flask 服务（若已安装）
    └─ 标准库 http.server 回退实现
  输出：浏览器中的动态图谱、查询结果、下载与脚本调度接口
```

如果按推荐顺序执行，完整项目流程应为：

```
# 1. 可选：检查环境
python scripts/check_deps.py
python scripts/check_torch.py

# 2. 数据采集与清洗
python src/data_extraction/run_extraction.py

# 3. NER 与实体消歧
python src/ner/batch_process.py --link True

# 4. 关系抽取、图构建与可视化
python scripts/run_pipeline.py

# 5. 可选：启动查询与前端界面
python scripts/run_webapp.py
```

说明：

- `scripts/run_pipeline.py` 目前默认只处理 5 篇示例文档，用于演示与课程作业汇报。
- 若要在全量数据上运行关系抽取，可分别执行 `src/relation_extraction/` 下的脚本并去掉文档子集限制。
- `scripts/run_webapp.py` 默认监听 `http://127.0.0.1:5000`；若环境中没有 Flask，会自动回退到标准库 HTTP 服务。

2.2 当前已完成模块的工作流程

下面的流程描述基于当前代码实现，而不是理想化设计。

模块	入口文件	主要输入	主要输出
环境检查	`scripts/check_deps.py` / `scripts/check_torch.py`	当前 Python 环境	控制台诊断信息
数据采集	`src/data_extraction/run_extraction.py`	`config/settings.yaml`	`data/raw/*.json`, `data/processed/*.json`
实体识别与消歧	`src/ner/batch_process.py`	`data/processed/*.json`	`output/entities_all.jsonl`
关系抽取	`scripts/run_pipeline.py` 或各子脚本	`data/processed/*.json`, `output/entities_all.jsonl`	`data/relation/*.jsonl`, `output/graphs/relation_triples*.jsonl`
图谱构建	`src/kg_construction/build_graph.py`	`relation_triples_aliased.jsonl`, `entities_all.jsonl`	`knowledge_graph.graphml/gexf/json`
可视化	`src/visualization/visualize.py`	`knowledge_graph.json`	`output/visualizations/*.png`, `*.html`
图谱查询 / Web 展示	`src/query/graph_query.py`, `scripts/run_webapp.py`	`knowledge_graph.json`	关键词检索结果、节点详情、局部子图、HTTP 接口与前端页面

2.2.1 数据采集模块工作流程

涉及文件：

- `src/data_extraction/run_extraction.py`

- `src/data_extraction/wiki_crawler.py`
- `src/data_extraction/wiki_parser.py`
- `src/data_extraction/data_cleaner.py`

执行流程：

1. `run_extraction.py` 读取 `config/settings.yaml` 中的 `crawler` 与 `paths` 配置。
2. `wiki_crawler.py` 以 `Alan Turing` 为种子做 BFS 爬取，控制最大深度、最大页数、请求间隔和相关性过滤。
3. `wiki_crawler.py` 对每个待处理页面调用 `wiki_parser.py`，后者通过 MediaWiki API 获取 HTML、摘要、分类和出链，并抽取章节文本与 Infobox。
4. 每个页面先写入 `data/raw/<title>.json`，作为保留原始结构的抓取结果。
5. `data_cleaner.py` 读取 `data/raw/*.json`，清洗引用标记、HTML 残留与多余空白，执行 Unicode NFKC 规范化并对摘要与章节分句。
6. 清洗后的结果写入 `data/processed/*.json`，供 NER 与关系抽取阶段直接使用。

模块输入输出：

- 输入：Wikipedia 页面标题、MediaWiki API 响应、`config/settings.yaml`。
- 输出：`data/raw/*.json` 与 `data/processed/*.json`。

与下游关系：

- `data/processed/*.json` 是 `src/ner/batch_process.py` 和 `src/relation_extraction/generate_candidates.py` 的共同上游。

2.2.2 实体识别与消歧模块工作流程

涉及文件：

- `src/ner/batch_process.py`
- `src/ner/spacy_ner.py`
- `src/ner/entity_linker.py`
- `src/ner/ner_pipeline.py`

执行流程：

1. `batch_process.py` 遍历 `data/processed/*.json`，通过 `extract_text_from_doc()` 提取摘要和章节正文。
2. `spacy_ner.py` 加载 spaCy 模型（默认 `en_core_web_sm`），识别每个文档中的实体 mention，输出字符级偏移和实体类型。
3. 如果命令行开启 `--link True`，`entity_linker.py` 会对每个 mention 调用 Wikidata 搜索 API，并使用向量相似度或启发式方法排序候选实体。
4. `batch_process.py` 对结果做裁剪和格式整理，只保留下游关系抽取需要的字段。
5. 所有文档结果最终汇总写入 `output/entities_all.jsonl`，每行对应一个文档。
6. `ner_pipeline.py` 提供面向少量文本的函数接口，适合调试，不负责目录级大规模处理。

模块输入输出：

- 输入：`data/processed/*.json` 中的文档文本。
- 输出：`output/entities_all.jsonl`，字段包括 `mention`、`type`、`start`、`end`、`wikidata_qid` 等。

与下游关系：

- `generate_candidates.py` 用它来定位句内实体对。
- `build_graph.py` 用它来补全节点的类型、QID 和描述。

2.2.3 关系抽取模块工作流程

涉及文件：

- `src/relation_extraction/extract_infobox_triples.py`
- `src/relation_extraction/generate_candidates.py`
- `src/relation_extraction/build_silver_labels.py`
- `src/relation_extraction/rebel_extract.py`
- `src/relation_extraction/merge_triples.py`
- `src/relation_extraction/apply_aliases.py`

执行流程：

1. `extract_infobox_triples.py` 从 `data/processed/*.json` 的 `infobox` 字段抽取高置信三元组，并通过 `config/relation_mapping.yaml` 做键名映射。
2. `generate_candidates.py` 将 `output/entities_all.jsonl` 中的实体与 `data/processed/*.json` 的句子边界对齐，在每个句子内生成实体对候选。
3. `build_silver_labels.py` 使用 Infobox 三元组作为远程监督信号，把候选对标成具体关系或 `none`，产出银标数据。
4. `rebel_extract.py` 加载 `Babelscape/rebel-large`，对候选对所在句子做生成式三元组抽取，并把模型输出重新对齐到候选实体 mention/QID。
5. `merge_triples.py` 对 REBEL 谓词做字面映射与 embedding 相似度映射，并按照 `Infobox > Silver > REBEL` 的优先级合并多源三元组。
6. `apply_aliases.py` 对最终三元组中的少量实体别名做标准化，当前主要解决 `Turing` 及其变体导致的重复节点问题。

模块输入输出：

- 输入：`data/processed/*.json`、`output/entities_all.jsonl`、`config/relation_mapping.yaml`。
- 输出：
  - `output/graphs/infobox_triples.jsonl`
  - `data/relation/candidates.jsonl`
  - `data/relation/silver.jsonl`
  - `output/graphs/rebel_triples.jsonl`
  - `output/graphs/relation_triples.jsonl`
  - `output/graphs/relation_triples_aliased.jsonl`

与下游关系：

- `build_graph.py` 默认使用 `relation_triples_aliased.jsonl` 作为建图输入。

2.2.4 知识图谱构建模块工作流程

涉及文件：

- `src/kg_construction/build_graph.py`

执行流程：

1. 读取最终三元组 JSONL。
2. 可选读取 `output/entities_all.jsonl`，建立 mention 到实体属性的映射。
3. 把三元组转换为 `networkx.DiGraph`，同时给节点补上类型、QID、描述等属性。
4. 对同一 `(head, tail, relation)` 的冲突边保留更高置信度版本。
5. 打印节点数、边数、关系分布、节点类型分布等统计信息。
6. 导出 GraphML、GEXF 和 JSON 三种格式，供分析与展示。

模块输入输出：

- 输入：`output/graphs/relation_triples_aliased.jsonl`，可选 `output/entities_all.jsonl`。
- 输出：`output/graphs/knowledge_graph.graphml`、`knowledge_graph.gexf`、`knowledge_graph.json`。

2.2.5 可视化模块工作流程

涉及文件：

- `src/visualization/visualize.py`

执行流程：

1. 从 `output/graphs/knowledge_graph.json` 读取图数据。
2. 使用 Matplotlib + NetworkX 生成组件感知布局的全图静态 PNG，按实体类型着色节点、按关系类型着色边，并为所有节点绘制标签。
3. 使用 pyvis 生成可交互 HTML，支持悬浮提示、拖拽和物理布局，边颜色与关系类型保持一致。
4. 以给定中心实体为核心提取 Ego 子图，同时生成对应的 PNG 和 HTML。

模块输入输出：

- 输入：`output/graphs/knowledge_graph.json`。
- 输出：`output/visualizations/full_graph.png`、`full_graph.html`、`ego_<center>.png`、`ego_<center>.html`。

三、技术方案

3.1 数据采集模块（Data Extraction）

3.1.1 目标

从英文维基百科出发，以 **Alan Turing** 页面为种子，按广度优先（BFS）策略爬取图灵本人及其关联实体（人物、机构、著作、事件等）的页面，提取 **非结构化文本** 与 **半结构化数据**。

3.1.2 技术选型

组件	技术	说明
HTTP 请求	`requests`	稳定的 HTTP 客户端，配合 `User-Agent` 和速率限制遵守 Wikipedia 爬虫协议
HTML 解析	`BeautifulSoup4`	解析 wiki 页面 DOM，提取正文段落、Infobox、分类等
Wikipedia API	`wikipedia-api`（Python 库）	通过 MediaWiki API 获取页面摘要、链接列表、分类等结构化数据
数据存储	JSON 文件	每个页面保存为一个 JSON 文件，包含标题、摘要、正文、Infobox、出链等字段

3.1.3 爬取流程

```
种子页面 "Alan Turing"
        │
        ▼
    ┌──────────────────────────┐
    │  wiki_crawler.py (BFS)   │
    │  ① 获取当前页面 HTML/API  │
    │  ② 提取页面内出链         │
    │  ③ 按相关性过滤出链       │
    │  ④ 入队，重复至 max_depth │
    └──────────────────────────┘
        │
        ▼
    ┌──────────────────────────┐
    │  wiki_parser.py          │
    │  ① 解析正文段落文本       │
    │  ② 提取 Infobox 键值对    │
    │  ③ 提取分类 (Categories)  │
    │  ④ 提取内部链接锚文本     │
    └──────────────────────────┘
        │
        ▼
    ┌──────────────────────────┐
    │  data_cleaner.py         │
    │  ① 去除引用标记 [1][2]    │
    │  ② 去除 HTML 残留标签     │
    │  ③ Unicode 规范化         │
    │  ④ 分句并保存             │
    └──────────────────────────┘
        │
        ▼
   data/raw/  →  data/processed/
```

3.1.4 关键设计

- **多跳 BFS 爬取**：`max_depth` 默认设为 2（即图灵 → 直接关联页面 → 二级关联页面），通过 `settings.yaml` 可配置。
- **相关性过滤**：仅保留与计算机科学 / 图灵相关的出链页面，过滤策略包括：
  - 排除"消歧义"、"文件:"、"模板:"等 Wikipedia 系统页面；
  - 基于种子关键词列表（Turing, computation, cryptography, AI, Bletchley Park 等）进行标题过滤；
  - 对超出关键词范围的链接可通过分类标签 (Category) 进行二次筛选。
- **请求速率限制**：遵循 Wikipedia `robots.txt`，每次请求间隔 ≥ 1 秒，设置合理的 `User-Agent`。
- **增量爬取**：已爬取页面记录在 `data/raw/` 中，再次运行时跳过已存在页面，支持断点续爬。
- **Infobox 提取**：从 Infobox 表格中提取键值对（如出生日期、国籍、母校、领域等），作为实体属性的重要来源。

3.1.5 输出格式

每个页面对应一个 JSON 文件 `data/raw/{page_title}.json`：

```
{
  "title": "Alan Turing",
  "url": "https://en.wikipedia.org/wiki/Alan_Turing",
  "summary": "Alan Mathison Turing was an English mathematician...",
  "sections": [
    {
      "heading": "Early life and education",
      "text": "Turing was born on 23 June 1912..."
    }
  ],
  "infobox": {
    "Born": "23 June 1912, Maida Vale, London",
    "Alma mater": "King's College, Cambridge; Princeton University",
    "Fields": "Mathematics, Computer science, Cryptanalysis"
  },
  "categories": ["British computer scientists", "Bletchley Park people"],
  "outlinks": ["Alonzo Church", "Bletchley Park", "Enigma machine"]
}
```

3.1.6 数据采集结果概况

以 `max_depth=2`、`max_pages=80` 运行后，实际采集数据如下：

指标	数值
爬取页面数	80
爬取层级分布	depth 0: 1 页, depth 1: 78 页, depth 2: 1 页
总章节数	961
总句子数	11,570
总出链数	27,728
清洗后数据量	4.3 MB

覆盖页面示例：Alan Turing、Turing Machine、Turing Test、Enigma machine、Bombe、Bletchley Park、Church–Turing thesis、Halting problem、Artificial intelligence、University of Cambridge、Princeton University 等。

3.2 实体识别与消歧模块

3.2.1 目标

从采集到的文本中自动识别实体（人物、地点、组织、概念、著作、装置等），并将同一现实世界实体的不同提及（mention）映射到唯一标识，消除歧义。

3.2.2 技术选型

组件	技术	说明
基础 NER	`spaCy`	Transformer / CNN 预训练模型，识别 PERSON、ORG、GPE、DATE 等通用实体
实体消歧	Wikidata API	将候选实体链接至 Wikidata QID，实现跨文档统一标识
指代消解	`spaCy` 实验性组件	将代词 (he/his/it) 还原为对应实体，提高下游召回率

3.2.3 实体消歧策略

1. **候选实体生成**：对每个识别出的 mention，调用 Wikidata Search API (`wbsearchentities`) 获取 Top-K 候选 QID。
2. **上下文相似度排序**：将 mention 的上下文窗口文本与各候选 QID 的 Wikidata 描述 (description) 计算余弦相似度（使用 `sentence-transformers` 或 TF-IDF），选取得分最高者。
3. **Infobox 属性辅助**：若 mention 来自 Infobox 字段（如 `Alma mater: King's College, Cambridge`），利用字段名语义直接约束候选类型。

4. **一致性约束**：同一文档中相同表面形式的 mention 映射到同一 QID。

3.2.4 实体类型体系

类型	标签	示例
人物	`PERSON`	Alan Turing, Alonzo Church, Max Newman
组织 / 机构	`ORG`	University of Cambridge, Bletchley Park, ACM
地点	`GPE`	London, Manchester, Princeton
概念 / 理论	`CONCEPT`	Turing Machine, Halting Problem, Artificial Intelligence
著作 / 系统	`WORK`	*On Computable Numbers*, ACE, Manchester Mark 1
装置	`DEVICE`	Enigma, Bombe
事件	`EVENT`	World War II
奖项 / 荣誉	`AWARD`	OBE, FRS, ACM Turing Award
日期	`DATE`	23 June 1912

3.2.5 输出格式

```
[
  {
    "mention": "Alan Turing",
    "type": "PERSON",
    "start": 0,
    "end": 11,
    "wikidata_qid": "Q7251",
    "link_confidence": 0.98,
    "source": "spacy"
  },
  {
    "mention": "Turing Machine",
    "type": "CONCEPT",
    "start": 156,
    "end": 170,
    "wikidata_qid": "Q163310",
    "link_confidence": 1.0,
    "source": "rule"
  }
]
```

3.2.6 批处理脚本与输出说明

项目提供一个可直接运行的批处理脚本 `src/ner/batch_process.py`，用于对 `data/processed/` 中的 JSON 文档批量执行 NER 与可选的实体消歧，并支持将结果合并为单个 JSONL 文件以便后续处理。主要特性：

- 支持 `--link` 参数开启/关闭 Wikidata 消歧（`--link True` 开启）。
- 支持 `--max-docs N` 仅处理前 N 个文档用于快速验证（例如 `--max-docs 1`）。
- 默认会把输出合并为 `output/entities_all.jsonl`（也可通过 `--output-dir` 指定其它路径）；合并写入使用临时文件 + `os.replace` 做原子替换，避免中间态损坏。
- 运行时在控制台显示 `tqdm` 进度条，并统计已识别实体数与已关联的实体数。
- 输出格式已精简：每个实体仅保留消歧后的选中项字段（`wikidata_qid`、`wikidata_label`、`wikidata_description`、`link_confidence`），不再写出完整的 `link_candidates` 列表（候选列表可在 `src/ner/entity_linker.py` 中查看或在开发时保留）。

示例用法：

```
# 只做 NER，不做消歧（快速）
python src/ner/batch_process.py --link False --max-docs 1

# 启用消歧并处理前5个文档
```

```
python src/ner/batch_process.py --link True --max-docs 5

# 全量运行（示例，耗时且会对 Wikidata 发出多次请求）
python src/ner/batch_process.py --link True
```

运行结果示例：`output/entities_all.jsonl` 中每一行为一个 JSON 对象，形式类似：

```
{"doc": "Alan Turing Year.json", "entities": [{"mention":"Alan Turing","type":"PERSON","start":91,"end":102,"source":"spacy","wikidata_qid":"Q7251","wikidata_label":"Alan Turing","link_confidence":1.0}, ...]}
```

3.3 关系抽取

实现文件：

脚本	功能
`src/relation_extraction/extract_infobox_triples.py`	Infobox 结构化三元组抽取
`src/relation_extraction/generate_candidates.py`	候选实体对生成
`src/relation_extraction/build_silver_labels.py`	远程监督银标构建
`src/relation_extraction/rebel_extract.py`	REBEL 模型三元组抽取与候选对对齐
`src/relation_extraction/merge_triples.py`	谓词归一化与多来源三元组合并
`src/relation_extraction/apply_aliases.py`	别名替换（最小化去重，例："Turing" → "Alan Turing"）

3.3.1 目标

从文本中抽取实体间的语义关系，形成 `(head, relation, tail)` 三元组。采用"Infobox 高精度 + 远程监督银标 + REBEL 模型扩展"的多来源融合策略，兼顾精度与覆盖率。

3.3.2 技术选型

组件	技术	说明
Infobox 抽取	规则映射	通过 `config/relation_mapping.yaml` 将 Infobox 键值对映射为标准关系，置信度 1.0
候选对生成	句内实体组合	对每个句子内的 NER 实体生成有序对，排除自身配对、双 DATE 和低价值类型
远程监督	Distant Supervision	用 Infobox 三元组匹配候选对，构建银标训练数据
文本抽取	`REBEL` (Babelscape/rebel-large)	预训练 Seq2Seq 模型，对候选句进行端到端三元组抽取
谓词归一化	字面映射 + `sentence-transformers` embedding 比对	将 REBEL 自由谓词文本映射到预定义关系标签
合并	多来源去重合并	按优先级 Infobox > Silver > REBEL 合并，保留 `provenance` 与 `confidence`

3.3.3 抽取流程

```
entities_all.jsonl + data/processed/*.json
        │
        ▼
┌──────────────────────────────────┐
│  extract_infobox_triples.py      │
│  ① 解析 Infobox 键值对           │
│  ② 按 relation_mapping.yaml 映射 │
│  ③ 输出 infobox_triples.jsonl    │
│     (confidence=1.0)             │
└──────────────────────────────────┘
```

```
        │
        ▼
┌──────────────────────────────────┐
│  generate_candidates.py          │
│  ① 按句内实体生成有序对           │
│  ② 过滤自身配对/双 DATE/低价值类型 │
│  ③ 每句最多 10 对                 │
│  ④ 输出 candidates.jsonl         │
└──────────────────────────────────┘
        │
        ├──────────────────────────┐
        ▼                          ▼
┌────────────────────┐  ┌──────────────────────────────┐
│  build_silver_labels │  │  rebel_extract.py            │
│  远程监督匹配 Infobox │  │  ① 去重获取唯一句子列表       │
│  → silver.jsonl      │  │  ② 加载 REBEL checkpoint     │
└────────────────────┘  │  ③ GPU fp16 批量推理 + tqdm  │
        │                 │  ④ 解析 REBEL 输出三元组      │
        │                 │  ⑤ 与候选对对齐（精确/子串）  │
        │                 │  ⑥ 替换为 NER/EL 实体信息     │
        │                 │  → rebel_triples.jsonl       │
        │                 └──────────────────────────────┘
        │                          │
        ▼                          ▼
┌──────────────────────────────────────────┐
│  merge_triples.py                        │
│  ① 谓词归一化（字面映射 + embedding 相似度）  │
│  ② 合并三来源（Infobox > Silver > REBEL）    │
│  ③ 按 (head, relation, tail) 去重          │
│  ④ 输出 relation_triples.jsonl             │
└──────────────────────────────────────────┘
```

3.3.4 关键设计

- **Infobox 优先**：来自 Infobox 的三元组置信度设为 1.0，在合并时具有最高优先级，保证高精度结构化知识直接入库。
- **REBEL 端到端抽取**：REBEL 模型以句子为输入，直接生成 `<triplet> subject <subj> object <obj> predicate` 格式文本，无需额外训练即可覆盖 Infobox 未涵盖的关系类型。
- **候选对对齐**：REBEL 输出的实体文本与候选对（来自 NER）做精确匹配或归一化子串匹配，匹配成功后将实体替换为 NER/EL 中的标准 mention 并附上 `wikidata_qid`，确保与上游实体识别结果一致。
- **谓词映射两阶段**：
  1. 字面映射：直接在 `config/relation_mapping.yaml` 中查找；
  2. Embedding 相似度映射：使用 `sentence-transformers/all-MiniLM-L6-v2` 计算 REBEL 谓词与预定义关系标签的余弦相似度，`≥ 0.65` 接受映射、`[0.55, 0.65)` 区间导出为模糊谓词供人工审查、`< 0.55` 保留原谓词文本。
- **OOM 自动降级**：REBEL 推理若遇显存不足，自动将 `batch_size` 减半（最多尝试 3 次），并在日志中记录降级理由。
- **审计与质检**：
  - 全程记录日志到 `output/logs/rebel_run.log`（环境信息、模型 checkpoint、batch size、fp16 启用与否、推理用时）；
  - 对 REBEL 输出抽样 50 条检查对齐质量，记录到 `output/logs/rebel_sample_qc.jsonl`；
  - 模糊谓词（相似度 `[0.55, 0.65)`）导出到 `output/logs/rebel_ambiguous_predicates.jsonl`。

3.3.5 输出格式

`output/graphs/relation_triples.jsonl`（合并后的最终三元组，每行一条 JSON）：

```
{
  "head": "Alan Turing",
  "head_qid": "Q7251",
  "relation": "educated_at",
  "tail": "King's College, Cambridge",
  "tail_qid": "Q5025572",
  "confidence": 1.0,
  "provenance": "infobox",
  "doc_ids": ["Alan Turing.json"],
  "sentence": "Alma mater: King's College, Cambridge"
}
```

3.3.6 运行结果概况（前 5 篇文档）

指标	数值
候选对总数	1,157
Infobox 三元组	21
Silver 正例三元组	43
REBEL 原始三元组	374
REBEL 对齐后三元组	149
REBEL 去重后三元组	132
合并后最终三元组	176
REBEL 推理用时	≈ 63s (RTX 3050, fp16, batch=4)

合并后三元组 provenance 分布：Infobox 21 条、Silver 25 条、REBEL 131 条。Top-10 关系类型：`authored_by` (15)、`part of` (10)、`notable work` (9)、`subject` (8)、`publication_date` (7)、`country` (7)、`employer` (7)、`birth_place` (6)、`point in time` (6)、`has part` (6)。

示例命令：

```
conda activate KG-Turing

# 一键运行完整关系抽取 + 图谱构建 + 可视化流程
python scripts/run_pipeline.py

# 或分步执行：
# 1. Infobox 三元组
python src/relation_extraction/extract_infobox_triples.py \
  --docs data/processed --mapping config/relation_mapping.yaml \
  --out output/graphs/infobox_triples.jsonl

# 2. 候选对生成
python src/relation_extraction/generate_candidates.py \
  --entities output/entities_all.jsonl --docs data/processed \
  --out data/relation/candidates.jsonl

# 3. 银标构建
python src/relation_extraction/build_silver_labels.py \
  --candidates data/relation/candidates.jsonl \
  --infobox output/graphs/infobox_triples.jsonl \
  --out data/relation/silver.jsonl

# 4. REBEL 抽取
python src/relation_extraction/rebel_extract.py \
  --candidates data/relation/candidates.jsonl \
  --out output/graphs/rebel_triples.jsonl \
  --model-name Babelscape/rebel-large --batch-size 8

# 5. 谓词归一化 & 合并
python src/relation_extraction/merge_triples.py \
```

```
  --rebel output/graphs/rebel_triples.jsonl \
  --infobox output/graphs/infobox_triples.jsonl \
  --silver data/relation/silver.jsonl \
  --mapping config/relation_mapping.yaml \
  --out output/graphs/relation_triples.jsonl
```

3.4 知识图谱构建与存储

实现文件：`src/kg_construction/build_graph.py`

要点：

- 使用 `networkx.DiGraph` 在内存中构建有向知识图谱，输入为关系三元组 JSONL（默认使用 `output/graphs/relation_triples_aliased.jsonl`）。
- 节点属性包含：`label`、`type`、`wikidata_qid`、`description`（若提供则从 `output/entities_all.jsonl` 中加载补全）；默认类型为 `UNKNOWN`。
- 边属性包含：`relation`、`confidence`、`sentence`、`doc`、`provenance`。构建时若遇到相同 `(head, tail, relation)` 的边，会保留置信度更高的一条。
- 提供图谱统计打印（节点数、边数、节点类型分布、关系分布、度数最高节点前 10 名），便于快速评估数据质量。
- 导出的 `knowledge_graph.json` 既是可视化输入，也是查询服务和 Web 前端的直接数据源。
- 输出多种持久化格式以便下游使用：
  - GraphML（保留所有属性）：`output/graphs/knowledge_graph.graphml`
  - GEXF（Gephi 兼容）：`output/graphs/knowledge_graph.gexf`
  - JSON（节点+边列表）：`output/graphs/knowledge_graph.json`

示例命令：

```
python src/kg_construction/build_graph.py \
  --triples output/graphs/relation_triples_aliased.jsonl \
  --entities output/entities_all.jsonl \
  --output-dir output/graphs
```

3.5 可视化

实现文件：`src/visualization/visualize.py`

已实现功能：

- **静态可视化（Matplotlib）**：采用组件感知布局与自适应画布尺寸，按实体类型着色节点、按关系类型着色边，节点大小按度数缩放，并为所有节点绘制标签，最终保存为 PNG（`output/visualizations/full_graph.png`）。
- **交互式可视化（pyvis）**：生成 HTML（`output/visualizations/full_graph.html`），包含节点 tooltip（显示 QID、描述、度数），支持节点拖拽、悬停提示与物理仿真布局，边颜色与关系类型保持一致。
- **Ego 子图**：按指定中心节点（默认 `Alan Turing`）和 radius（默认 2）同时生成静态 PNG 与交互式 HTML（文件名 `ego_{center}.png` / `.html`），便于聚焦某一实体的局部网络。

主要实现细节：

- Matplotlib 使用无头后端 `Agg` 以便在服务器/CI 环境生成图片。
- 静态图会为高频关系生成图例，并通过组件布局尽量保留小连通分量的可见性。
- pyvis 配置了物理引擎参数（forceAtlas2Based）以改善布局收敛性，并为每个节点构建富文本 tooltip（包含 `wikidata_qid`、`description` 与 degree）。

示例命令：

```
python src/visualization/visualize.py \
  --graph output/graphs/knowledge_graph.json \
  --output-dir output/visualizations \
  --ego-center "Alan Turing" \
  --ego-radius 2
```

3.6 图谱查询

实现文件：

- `src/query/graph_query.py`
- `src/webapp/app.py`
- `scripts/run_webapp.py`
- `src/webapp/templates/index.html`
- `src/webapp/static/app.js`
- `src/webapp/static/style.css`

已实现能力：

- **关键词检索**：基于 `knowledge_graph.json` 建立节点索引，支持按实体名、QID、类型和描述做打分匹配。
- **节点详情**：返回节点属性、度数、入边、出边以及相关邻居信息。
- **局部子图提取**：可按中心节点和半径生成子图，直接服务前端动态图谱展示。
- **图谱概览**：支持统计节点数、边数、主要实体类型和主要关系类型。
- **Web API 与前端界面**：提供动态图谱、关键词查询、产物下载、白名单脚本执行与任务轮询。

运行方式：

```
python scripts/run_webapp.py
```

说明：

- 默认访问地址为 `http://127.0.0.1:5000`。
- 若环境中安装了 Flask，则优先使用 Flask 提供服务；否则自动回退到标准库 `http.server`。
- 当前查询层直接消费 `output/graphs/knowledge_graph.json`，不依赖关系抽取阶段的中间文件。

四、环境要求

- **Python**: 推荐 3.10.x（在本环境中测试为 3.10.20）。
- **Conda 环境**: 建议使用名为 `KG-Turing` 的虚拟环境以便复现。

安装与验证（推荐顺序）：

```
# 1) 创建并激活 Conda 环境
conda create -n KG-Turing python=3.10 -y
conda activate KG-Turing

# 2) PyTorch（必须/按是否有 GPU 选择）
#  - 无 GPU（CPU-only）示例：
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cpu
#  - 有 NVIDIA GPU：请按你的 CUDA 版本从 https://pytorch.org/ 获取对应命令。
#    示例（CUDA 11.8）：
pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118

# 3) Transformers + 加速器（REBEL 依赖 transformers）
pip install transformers accelerate

# 4) 其它 Python 依赖（在安装好 torch/transformers 之后）
pip install -r requirements.txt

# 5) spaCy 英文模型
python -m spacy download en_core_web_sm
# （可选）若使用 transformer-backed spaCy 模型：
# python -m spacy download en_core_web_trf
```

快速环境验证：

```
python -c "import sys, torch; print('Python', sys.version.split()[0]); print('torch', torch.__version__);
```

```
print('CUDA available:', torch.cuda.is_available())"
```

也可以直接使用仓库内置的检查脚本：

```
python scripts/check_deps.py
python scripts/check_torch.py
```

若需要使用浏览器界面，可在图谱文件生成后执行：

```
python scripts/run_webapp.py
```

注意：

- REBEL/transformers 会在首次运行时自动下载模型文件，确保有足够的磁盘空间和网络带宽。
- 若使用 GPU，请先安装合适的 NVIDIA 驱动与 CUDA 运行时，并根据系统选择与 `torch` 兼容的 CUDA 版本。
- 在 Windows 上安装部分底层依赖可能需要 Microsoft Visual C++ Redistributable / Build Tools。
- `scripts/check_deps.py` 还会检查 `pyvis` 以及可选依赖 `flask`、`pytest`；其中 `flask` 缺失不会阻止前端启动，因为项目会自动回退到标准库 HTTP 服务。

五、主要依赖

包名	用途
`requests`	HTTP 请求
`beautifulsoup4`	HTML 解析
`wikipedia-api`	Wikipedia 结构化数据获取
`spacy`	NER、依存解析、实体链接
`torch`	REBEL 推理与 GPU/CPU 计算
`networkx`	图结构构建与算法
`matplotlib`	静态图谱可视化
`pyvis`	交互式图谱 HTML 可视化
`pyyaml`	配置文件解析
`lxml`	HTML / XML 解析加速
`sentence-transformers`（可选）	实体消歧上下文相似度计算
`transformers`	REBEL 模型与序列到序列推理
`accelerate`	模型加速与分布式推理支持
`tqdm`	控制台进度条、批处理可视化
`flask`（可选）	Web 服务运行时
`pytest`（测试）	查询层与 Web API 测试

六、开发进度

阶段	模块	状态
Phase 1	数据采集	✅ 已完成
Phase 2	实体识别与消歧	✅ 已完成
Phase 3	关系抽取	✅ 已完成
Phase 4	知识图谱构建	✅ 已完成

阶段	模块	状态
Phase 5	可视化	✅ 已完成
Phase 6	查询 / Web 前端	✅ 已完成

七、参考资料

- Alan Turing - Wikipedia
- Wikidata - Alan Turing (Q7251)
- spaCy Documentation
- NetworkX Documentation
- Wikipedia API (MediaWiki)